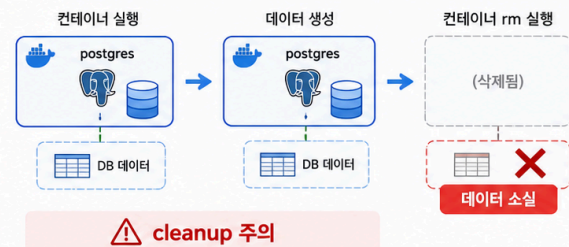


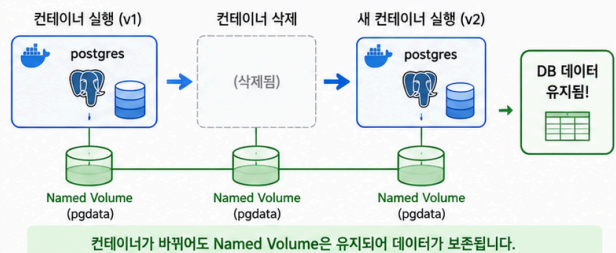
1교시: Day 1 DB container 재생성과 데이터 소실 확인

Day 2: Docker Storage & Network

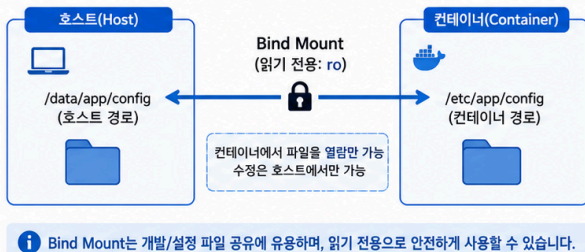
1 Volume 없이 PostgreSQL 컨테이너 삭제 시 데이터 소실



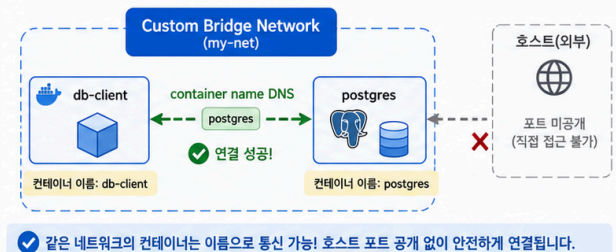
2 Named Volume 사용 시 컨테이너 교체 후에도 데이터 보존



3 Bind Mount로 호스트 경로와 컨테이너 경로 연결 (읽기 전용 옵션)



4 Custom Bridge Network에서 컨테이너 이름으로 통신 (호스트 포트 미공개)



수업 목표

- Day 1에서 volume 없이 만든 PostgreSQL data가 container lifecycle에 묶였는지 확인한다.
- container 삭제와 database data 삭제의 관계를 실험으로 설명한다.
- 데이터 소실을 Day 2 volume 학습의 출발점으로 삼는다.

강의 전개

Day 1에서 SQL로 user, database, table, row를 만들었다면 학생은 그것이 Docker 어딘가에 안전하게 저장되었다고 착각하기 쉽다. 이 교시는 그 착각을 의도적으로 깨는 시간이다. volume 없이 만든 database container를 삭제하고 같은 image로 다시 만들면, 이전에 만든 table과 row가 보이지 않는 상황을 확인한다. 핵심은 PostgreSQL이 나쁜 것이 아니라 container writable layer와 data lifecycle을 분리하지 않았다는 점이다.

이 교시는 설명만 듣고 지나가지 않는다. 명령은 반드시 code block으로 실행하고, 바로 이어서 검증 명령을 실행한다. 정상 출력이 다를 수 있는 부분은 전체 문자열을 외우지 않고 성공 패턴을 기록한다. 실패도 수업 산출물이다. 실패한 명령, 에러 요약, 가설, 다시 확인한 명령을 함께 남긴다.

실습 명령

```
docker ps -a --filter name=paperclip-pg16
docker stop paperclip-pg16 || true
docker rm paperclip-pg16 || true
docker run -d --name paperclip-pg16 -e POSTGRES_PASSWORD=postgres -e
POSTGRES_DB=paperclip -p 15432:5432 postgres:16
```

검증 명령

```
docker logs paperclip-pg16 --tail 30
docker exec paperclip-pg16 psql -U postgres -d paperclip -c "\dt"
```

실패 드릴과 오해 교정

상황	해석
table이 없다	volume 없이 새 container를 만들었으므로 이전 writable layer data가 사라진 상태다.
container가 실행되지 않는다	이름 충돌, port 충돌, POSTGRES_PASSWORD 누락을 먼저 확인한다.
5432 접속이 안 된다	host port 15432와 container port 5432를 구분한다.

Cleanup

```
docker stop paperclip-pg16 || true
docker rm paperclip-pg16 || true
```

Cleanup은 비용과 데이터 안전을 동시에 다룬다. container를 지우는 명령과 volume/network/image를 지우는 명령은 의미가 다르다. 특히 volume 삭제는 database data 삭제일 수 있으므로 실습 volume인지 확인한 뒤 실행한다.

Evidence

항목	제출 기준
No-volume 결과	이전 table/row가 새 container에서 보이지 않는 query 결과
명령 기록	run/check/cleanup 명령
해석	왜 데이터가 사라졌는지 한 문장

강의자 설명 포인트

이 실습의 핵심은 명령어 자체가 아니라 경계다. container는 실행 단위이고, volume은 data lifecycle이며, network는 통신 경계다. 학생이 docker run 한 줄을 볼 때 -v, --network, -p를 옵션 목록으로 외우면 뒤에서 Compose와 Kubernetes로 넘어갈 때 같은 혼란이 반복된다. 그래서 각 옵션을 "무엇을 container 밖으로 분리하는가"라는 질문으로 읽게 한다.

강의 중에는 성공 출력보다 실패 출력의 의미를 더 오래 다룬다. port가 열리지 않은 것은 web server 문제가 아닐 수 있고, DB 접속 실패는 password 문제가 아니라 network boundary 문제일 수 있다. host terminal, container 내부, 같은 Docker network의 client container는 모두 서로 다른 관찰 위치다. 학생이 어디에서 명령을 실행하는지 말로 먼저 설명한 뒤 CLI를 실행하게 한다.

운영 해석

실무에서 database container를 다룰 때 가장 위험한 실수는 cleanup을 단순 정리로만 보는 것이다. container 삭제는 process와 container writable layer를 정리하는 것이고, volume 삭제는 data를 삭제하는 것이다. network 삭제는 통신 경로를 정리하는 것이다. 이 세 가지를 구분하지 않으면 실습은 성공해도 운영 사고를 배운 셈이 된다.

README에는 "실행됐다" 대신 "어떤 data를 남기고 무엇을 삭제했는지"를 써야 한다. Day 2 evidence는 Day 5 Compose에서 volumes와 networks를 읽는 기준이 된다. Compose의 YAML 항목은 갑자기 생긴 문법이 아니라 Day 2에서 손으로 실행한 storage/network 결정을 파일로 옮긴 것이다.

README 기록 예시

```
## Storage/Network Evidence
- Container:
- Volume name:
- Volume target path:
- Network name:
- Host port published? yes/no
- Container DNS check:
- Data survived container replacement? yes/no
- Cleanup decision:
```

다음 연결

다음 교시는 named volume을 만들어 같은 실험을 반복한다. 목표는 container를 교체해도 data가 남는 구조를 만드는 것이다.